



Uni. Tec. Nac

TPI

Gestion de datos de países en python

Programacion 1

Michel, Renzo
Rodriguez, Santiago

Tec. en programacion

Entrega: 22/06

Índice

1. Carátula	1
2. Índice	2
4. Marco teórico	3 - 4
5. Decisiones técnicas y arquitectura	5 - 6
6. Dificultades y conclusiones	7
7. Bibliografía / webgrafía	8
8. Links al repositorio y al video explicativo	8

Marco Teórico

Este proyecto consiste en el desarrollo de un sistema de gestión de países utilizando el lenguaje de programación Python. Se usa para administrar información almacenada en un archivo CSV, permitiendo realizar funciones como agregar países, actualizar datos, realizar búsquedas, aplicar filtros, ordenar registros y obtener estadísticas sobre la información almacenada.

Para la implementación del sistema se aplicaron conceptos de manipulación de archivos, la persistencia de datos y el uso de estructuras de datos. Los registros fueron almacenados y gestionados mediante diccionarios, mientras que las listas fueron utilizadas para operaciones de filtrado, ordenamiento y procesamiento de información. Esta combinación permitió organizar los datos de manera eficiente y facilitar el acceso a cada uno de los atributos de los países registrados.

Además, se utilizaron técnicas de lectura y procesamiento de archivos CSV mediante las funciones `strip()` y `split()`, las cuales permitieron interpretar correctamente la información almacenada en el archivo. También se implementaron validaciones de datos y manejo de excepciones para garantizar la integridad de la información ingresada por el usuario y mejorar la robustez del sistema.

A lo largo de este informe se describen los conceptos teóricos aplicados, las decisiones técnicas tomadas durante el desarrollo, la arquitectura de la solución implementada y las conclusiones obtenidas a partir de la realización del proyecto.

Por eso se recurre al almacenamiento persistente, que permite conservar información entre ejecuciones, intercambiar datos con otros sistemas y mantener registros de forma ordenada. Un archivo se define como una unidad de almacenamiento que contiene datos que pueden ser leídos o escritos por un programa. Dentro de los materiales analizados se distinguen principalmente archivos de texto plano (.txt) y archivos CSV (.csv).

Los CSV (que usamos en este trabajo) son especialmente útiles porque representan información en forma de tabla, donde cada línea funciona como una fila y cada valor separado por comas o por otro delimitador actúa como una columna. Para abrir archivos en Python se utiliza `open()`, indicando el nombre del archivo y el modo de apertura. Los apuntes describen los modos más usados: 'r' para lectura, 'w' para escritura, y 'a' para agregar contenido al final sin borrar lo anterior. También se destaca la conveniencia de usar `with`, ya que garantiza el cierre automático del archivo al terminar el bloque. En la lectura de archivos se mencionan operaciones básicas como recorrer línea por línea con "for", eliminar espacios y saltos de línea con `strip()`, y separar campos con `split()` que se usa repetidamente en el proyecto.

En el proyecto también se utilizó `encoding="utf-8"` al abrir el archivo CSV, con el fin de garantizar la correcta lectura y escritura de caracteres especiales, como tildes y la letra ñ, evitando errores de interpretación de los datos.

En el proyecto aplicamos:

La persistencia de datos mediante un archivo CSV como fuente inicial de información, uso de estructuras de datos de Python para representar cada país como un diccionario anidado lectura manual del CSV combinando `with`, `next()`, `strip()` y `split()` y validación de entrada con excepciones y bucles de repetición.

Decisiones Técnicas y Arquitectura

El proyecto se organizó en tres archivos, dos archivos de Python y uno CSV. El archivo main actúa como punto de entrada: importa el módulo funciones, carga los países desde el archivo CSV y mantiene un bucle principal que muestra el menú y deriva cada opción a su función correspondiente. El archivo funciones concentra la lógica del sistema para mantener el código ordenado y modular.

La información se guarda en un diccionario principal llamado “países”. Cada clave es el nombre del país y el valor asociado es otro diccionario con tres campos: Continente, Población y Superficie. Esta elección facilita búsquedas, actualizaciones y filtrados por atributos.

Diagrama de flujo

El funcionamiento del sistema es simple, desde el inicio está el menú, que inicialmente carga el archivo csv, una vez el archivo contiene algo, se puede usar el sistema usando una función, valida la entrada ingresada y al terminar la función vuelve al menú de opciones, con el uso de un match-case para moverse por el menú.

```
def menu_opciones():
    print('''----- Menu -----
1. Agregar un pais
2. Actualizar datos (Poblacion y superficie)
3. Buscar un pais por nombre
4. Filtrar paises (Continente, rango de poblacion y rango de superficie)
5. Ordenar paises (Nombre, poblacion y superficie)
6. Mostrar estadísticas
7. Salir.
''')
    opcion = input('Elija su opcion: ')
    return opcion
```

Cada opción, incluyendo el menú es una función en el código, y esta función muestra el menú, actúa como la interfaz principal de interacción entre el usuario y el sistema, presentando de forma clara y organizada las distintas funcionalidades disponibles, permitiendo que el usuario navegue por el programa de manera intuitiva mediante el menú de opciones.

```
def cargar_paises():
    paises_agregados = {}
    with open("paises.csv", "r", encoding="utf-8") as archivo:
        next(archivo)
        for i in archivo:
            paises = i.strip().split(',')
            nombre = paises[0]
            continente = paises[1]
            poblacion = int(paises[2])
            superficie = int(paises[3])
            #Se crea el diccionario paises_agregados con un diccionario como valor de cada uno
            paises_agregados[nombre] = {'Continente': continente, 'Poblacion': poblacion, 'Superficie': superficie}
    return paises_agregados
```

Una vez muestra el menú se ejecuta la función cargar países tomando la información del archivo CSV y se divide la información en una lista con la ayuda de `.strip()` y `.split()` para luego ingresarlo a un diccionario que vuelve al archivo main listo para comenzar a utilizar las opciones del sistema.

Dificultades y conclusiones

Uno de los principales desafíos del proyecto fue trabajar con datos cargados desde un archivo CSV y convertirlos en una estructura de Python que luego pudiera actualizarse, filtrarse y ordenarse. La solución fue leer el archivo línea por línea, limpiar cada registro con `strip()`, separarlo con `split(',')` y guardar cada país en un diccionario anidado.

Otra dificultad fue evitar entradas inválidas del usuario. Para resolverlo usamos bucles `while True` junto con `try/except`, validando nombres, continentes, poblaciones y superficies antes de modificar la información. Además, las excepciones personalizadas permitieron mostrar mensajes más claros para el usuario.

Desde el punto de vista del aprendizaje, el trabajo nos permitió integrar conceptos de archivos, estructuras de datos, funciones y validación de datos. La organización del programa en módulos también ayudó a mantener el código más entendible y a separar responsabilidades entre la interfaz del menú y la lógica de cada operación.

Como conclusión general, el proyecto demuestra cómo un archivo CSV puede convertirse en una base de datos simple para una aplicación de consola, y cómo Python permite manipular esa información de manera ordenada mediante funciones, diccionarios y manejo de errores.

Bibliografía / Webgrafía

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57197/mod_label/intro/1%20-%20Introducci%C3%B3n%20Archivos%20.pdf

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57199/mod_folder/content/0/2%20-%20Apertura%2C%20lectura%2C%20escritura%2C%20cierre%20de%20archivos.pdf?forcedownload=1

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57201/mod_folder/content/0/3%20-%20strip%28%29%2C%20split%28%29%2C%20.join%28%29.docx.pdf?forcedownload=1

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57202/mod_label/intro/5%20-%20Manejo%20de%20errores%20con%20archivos.docx.pdf

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57183/mod_label/intro/Manejo%20de%20Errores_%20Tipo%20de%20Errores.docx.pdf

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57184/mod_label/intro/Manejo%20de%20Errores_%20Try%20Except.docx.pdf?time=1771449850438

https://campusvirtual.frm.utn.edu.ar/pluginfile.php/57171/mod_label/intro/Diccionarios%20-%20TUPaD.docx%20%281%29.pdf

Links al Repositorio y Video Explicativo

Repositorio

https://github.com/Ram3rr/Programacion_1_TPI

Video

<https://youtu.be/iPHs-zWbmZ4>